LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 6



IMMANUEL RADITYA DEO PRATAMA PAMBUDHI

103132430001

S1SD-05-A

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

Dasar Teori

1. Konsep Dasar Inheritance (Pewarisan)
Inheritance adalah pilar OOP yang memungkinkan suatu kelas (subclass) mewarisi atribut dan method dari kelas lain (superclass) dengan kata kunci extends. Relasi ini mencerminkan hubungan 'is-a' (adalah jenis dari). Manfaat utamanya adalah mencegah duplikasi kode (code reusability) serta menstrukturkan kelas dari bentuk umum ke bentuk yang lebih spesifik.

2. Keyword super
Kata kunci super berfungsi untuk mereferensi anggota milik superclass dari dalam subclass. Dalam constructor subclass, super(...) digunakan untuk memanggil constructor superclass agar bagian objek kelas induk terinisialisasi lebih dahulu. Selain itu, super.method() atau super.variabel dapat digunakan untuk mengakses method atau atribut superclass yang tertimpa di subclass.

3. Overloading Method
Overloading adalah pembuatan beberapa method dengan nama yang sama di dalam satu kelas atau hierarki kelas, dengan syarat jumlah, tipe, atau urutan parameternya berbeda. Pemanggilan method yang sesuai ditentukan secara otomatis saat waktu kompilasi (compile-time polymorphism) berdasarkan argumen yang dikirimkan.

4. Overriding Method
Overriding adalah penulisan ulang method superclass di dalam subclass dengan nama, tipe kembalian, dan parameter yang identik. Tujuannya adalah memberikan implementasi perilaku yang lebih spesifik bagi subclass. Ketika method dipanggil melalui instance subclass, method versi subclass-lah yang dieksekusi, yang mendasari konsep polimorfisme runtime.

Guided

Guided 1: Inheritance Dasar (A.java, B.java, dan Demo.java)

A.java:

```java
package guided.gd1;

public class A {
    int x;
    int y;

    void tampilXY() {
        System.out.println("nilai x: " + x + " nilai y: " + y);
    }
}
```

B.java:

```java
package guided.gd1;

public class B extends A{
    int z;

    void jumlahXY(){
        System.out.println("Jumlah (x+y+z) = " + (x + y + z));
    }
}
```

Demo.java:

```java
package guided.gd1;

public class Demo {
    public static void main(String[] args) {
        A superclass = new A();
        B subclass = new B();
```

```
        superclass.x = 3;
        superclass.y = 4;

        System.out.println("Superclass: ");
        superclass.tampilXY();

        subclass.x = 1;
        subclass.y = 2;
        subclass.z = 5;

        System.out.println("Subclass: ");
        subclass.jumlahXY();
        subclass.tampilXY();
    }
}
```

Output :

```
C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-06-inheritance\project\src\main\java>java guided.gd1.Demo
Superclass:
nilai x: 3 nilai y: 4
Subclass:
Jumlah (x+y+z) = 8
nilai x: 1 nilai y: 2

C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-06-inheritance\project\src\main\java>_
```

Guided 2: Keyword super (Employee.java, Manager.java, dan Demo.java)

Employee.java:

```
package guided.gd2;

public class Employee {
    private String name;
    String departement;

    public Employee(String name){
```

```
        this.name = name;
    }

    public void tampilNama(){
        System.out.println("nama: " + name);
    }
}
```

Manager.java:

```
package guided.gd2;

public class Manager extends Employee{
    private String alamat;

    public Manager(String name, String alamat){
        super(name);
        this.alamat = alamat;
    }

    public void tampilInformasi(){
        super.tampilNama();
        System.out.println("alamat: " + alamat);
        System.out.println("departemen: " + super.departement);
    }
}
```

Demo.java:

```
package guided.gd2;

public class Demo {
    public static void main(String[] args) {
        Manager adi = new Manager("Adi", "Sukabirus");
        adi.departement = "SDM";
        adi.tampilInformasi();
```

```
        }
    }
```

Output :

```
C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-06-inheritance\project\src\main\java>java guided.gd2.Demo
nama: Adi
alamat: Sukabirus
departemen: SDM

C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-06-inheritance\project\src\main\java>_
```

Guided 3: Overloading Method (Lagu.java dan Demo.java)

Lagu.java:

```java
package guided.gd3;

public class Lagu {
    String pencipta;
    String judul;

    void IsiParam(String param1) {
        judul = param1;
        pencipta = "Tidak dikenal";
    }

    void IsiParam(String param1, String param2) {
        judul = param1;
        pencipta = param2;
    }

    void tampilkan() {
        System.out.println("Judul : " + judul + ", pencipta : " + pencipta);
    }
}
```

Demo.java:

```java
package guided.gd3;

public class Demo {
    public static void main(String[] args) {
        Lagu d, e;
        d = new Lagu();
        e = new Lagu();

        d.IsiParam("Lagu 1");
        e.IsiParam("Lagu 2", "aku");

        d.tampilkan();
        e.tampilkan();
    }
}
```

Output :

```
C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-06-inheritance\project\src\main\java>java guided.gd3.Demo
Judul : Lagu 1, pencipta : Tidak dikenal
Judul : Lagu 2, pencipta : aku

C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-06-inheritance\project\src\main\java>_
```

Guided 4: Overriding Method (A.java, B.java, dan Demo.java)

A.java:

```java
package guided.gd4;

public class A {
    public void tampilkan(){
        System.out.println("Method milik class A dipanggil...");
    }
}
```

B.java:

```java
package guided.gd4;

public class B extends A{
    public void tampilkan(){
        super.tampilkan();
        System.out.println("Method milik class B dipanggil...");
    }
}
```

Demo.java:

```java
package guided.gd4;

public class Demo {
    public static void main(String[] args) {
        B  overridB = new B();
        overridB.tampilkan();
    }
}
```

Output :

```
C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-06-inheritance\project\src\main\java>java guided.gd4.Demo
Method milik class A dipanggil...
Method milik class B dipanggil...

C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-06-inheritance\project\src\main\java>_
```

Unguided

Soal Unguided 1

Sebuah tim machine learning ingin membuat program untuk membandingkan estimasi biaya komputasi dari beberapa jenis model yang akan dilatih. Ada tiga jenis model: model regresi, model klasifikasi, dan model klasifikasi berbasis deep learning yang merupakan pengembangan dari model klasifikasi biasa. Program wajib menerapkan enkapsulasi penuh, inheritance berjenjang, overloading, overriding, serta pemanggilan super constructor.

Jawaban :

ModelML.java:

```
package unguided;

public class ModelML {
    private String namaModel;
    private int jumlahDataLatih;

    // Constructor
    public ModelML(String namaModel, int jumlahDataLatih) {
        this.namaModel = namaModel;
        this.jumlahDataLatih = jumlahDataLatih;
    }

    // Behavior
    public void tampilInformasi() {
        System.out.println("Model: " + this.namaModel + " | Data latih: " +
this.jumlahDataLatih);
    }

    public int hitungBiaya() {
        return this.jumlahDataLatih * 2;
    }

    public int hitungBiaya(int epoch) {
        return this.jumlahDataLatih * epoch;
    }
```

```
    // Getter
    public String getNamaModel() {
        return this.namaModel;
    }

    public int getJumlahDataLatih() {
        return this.jumlahDataLatih;
    }

    // Setter
    public void setNamaModel(String namaModel) {
        this.namaModel = namaModel;
    }

    public void setJumlahDataLatih(int jumlahDataLatih) {
        this.jumlahDataLatih = jumlahDataLatih;
    }
}
```

ModelRegresi.java:

```
package unguided;

public class ModelRegresi extends ModelML {
    private int jumlahFitur;

    // Constructor
    public ModelRegresi(String namaModel, int jumlahDataLatih, int jumlahFitur)
{
        super(namaModel, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
    }

    // Behavior
    public void tampilInformasi() {
        super.tampilInformasi();
```

```
        System.out.println("Jumlah fitur: " + this.jumlahFitur);
    }

    public int hitungBiaya() {
        return super.hitungBiaya() + (this.jumlahFitur * 10);
    }

    public int hitungBiaya(int epoch, int batchSize) {
        return (this.getJumlahDataLatih() * epoch) / batchSize;
    }

    // Getter
    public int getJumlahFitur() {
        return this.jumlahFitur;
    }

    // Setter
    public void setJumlahFitur(int jumlahFitur) {
        this.jumlahFitur = jumlahFitur;
    }
}
```

ModelKlasifikasi.java:

```
package unguided;

public class ModelKlasifikasi extends ModelML {
    private int jumlahKelas;

    // Constructor
    public ModelKlasifikasi(String namaModel, int jumlahDataLatih, int
jumlahKelas) {
        super(namaModel, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }
```

```
    // Behavior
    public void tampilInformasi() {
        super.tampilInformasi();
        System.out.println("Jumlah kelas: " + this.jumlahKelas);
    }

    public int hitungBiaya() {
        return super.hitungBiaya() * this.jumlahKelas;
    }

    // Getter
    public int getJumlahKelas() {
        return this.jumlahKelas;
    }

    // Setter
    public void setJumlahKelas(int jumlahKelas) {
        this.jumlahKelas = jumlahKelas;
    }
}
```

ModelDL.java:

```
package unguided;

public class ModelDL extends ModelKlasifikasi {
    private int jumlahLayer;

    // Constructor
    public ModelDL(String namaModel, int jumlahDataLatih, int jumlahKelas, int
jumlahLayer) {
        super(namaModel, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    // Behavior
```

```
    public void tampilInformasi() {
        super.tampilInformasi();
        System.out.println("Jumlah layer: " + this.jumlahLayer);
    }

    public int hitungBiaya() {
        return super.hitungBiaya() + (this.jumlahLayer * 200);
    }

    // Getter
    public int getJumlahLayer() {
        return this.jumlahLayer;
    }

    // Setter
    public void setJumlahLayer(int jumlahLayer) {
        this.jumlahLayer = jumlahLayer;
    }
}
```

Main.java:

```
package unguided;

public class Main {
    // Behavior
    public static void main(String[] args) {
        ModelML[] daftarModel = new ModelML[3];
        daftarModel[0] = new ModelRegresi("Regresi Linier", 500, 4);
        daftarModel[1] = new ModelKlasifikasi("Decision Tree", 800, 3);
        daftarModel[2] = new ModelDL("Neural Network", 1000, 5, 3);

        int totalBiaya = 0;
        for (ModelML model : daftarModel) {
            model.tampilInformasi();
            int biaya = model.hitungBiaya();
```

```
            System.out.println("Biaya komputasi: " + biaya);
            System.out.println("---");
            totalBiaya += biaya;
        }

        System.out.println("Total biaya: " + totalBiaya);
        System.out.println("Biaya 10 epoch (Neural Network): " +
daftarModel[2].hitungBiaya(10));
        System.out.println("Biaya 10 epoch (Regresi Linier): " +
daftarModel[0].hitungBiaya(10));
        System.out.println("Biaya 10 epoch batch 50 (Regresi Linier): " +
((ModelRegresi) daftarModel[0]).hitungBiaya(10, 50));
    }
}
```

Output :

```
C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-06-inheritance\project\src\main\java>java unguided.Main
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100

C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-06-inheritance\project\src\main\java>_
```

Penjelasan :

Pada tugas unguided ini, aku membuat sistem perhitungan estimasi biaya komputasi machine learning dengan menerapkan konsep pewarisan (inheritance) berjenjang, pemanggilan super constructor, overloading, dan overriding method:

Pewarisan Kelas: Kelas ModelML menjadi kelas induk utama yang menyimpan atribut private namaModel dan jumlahDataLatih. Kelas ModelRegresi dan ModelKlasifikasi mewarisi langsung ModelML dengan kata kunci extends. Sedangkan ModelDL merupakan turunan berjenjang (multilevel inheritance) dari ModelKlasifikasi, sehingga otomatis membawa sifat dari ModelKlasifikasi sekaligus ModelML.
Penggunaan super: Setiap kelas turunan memanfaatkan super(...) di constructor-nya untuk meneruskan parameter ke kelas di atasnya, serta memanggil super.tampilInformasi() dan super.hitungBiaya() agar logika dasar dari superclass tetap dijalankan sebelum ditambahkan komponen biaya spesifik.
Overriding dan Overloading: Method hitungBiaya() tanpa parameter di-override di setiap subclass (ModelRegresi menambah biaya fitur, ModelKlasifikasi mengalikan jumlah kelas, dan ModelDL menambah biaya layer). Di saat yang sama, overloading terjadi pada method hitungBiaya(int epoch) serta method baru dengan dua parameter hitungBiaya(int epoch, int batchSize) di kelas ModelRegresi.
Polimorfisme di Main: Ketiga objek model ditampung dalam satu array polimorfik bertipe ModelML[]. Saat dilakukan looping, Java memanggil method hitungBiaya() versi kelas aslinya masing-masing secara dinamis, lalu seluruh biaya dijumlahkan dan dihitung variasi biaya per epoch serta batch size-nya dengan hasil yang presisi.

Kesimpulan

Berdasarkan praktikum modul 6 yang telah diselesaikan, beberapa kesimpulan penting yang aku peroleh adalah:
1. Pewarisan (Inheritance): Konsep extends memungkinkan subclass mewarisi field dan method superclass sehingga mencegah penulisan kode berulang dan menciptakan hierarki program yang terstruktur rapi.
2. Keyword super: Sangat krusial untuk menghubungkan subclass dengan superclass, baik untuk memicu constructor induk maupun memanggil implementasi method asli sebelum diperluas.

3. Overloading vs Overriding: Overloading membedakan method berdasarkan parameter dalam kelas/hierarki yang sama, sedangkan overriding menimpa logika method induk di kelas anak agar perilakunya sesuai kebutuhan khusus.
4. Polimorfisme: Memungkinkan berbagai objek dari turunan berbeda dikelola secara seragam melalui tipe kelas induk (seperti ModelML[]), memudahkan otomasi pemrosesan data.